buildandteach

PYSPARK QUICK REFERENCE

Join safely: define the grain

Orders: one row per order. Lookup: one row per customer.
Inner keeps matches. Left retains unmatched orders, but may multiply matches.

1. Check lookup keys BEFORE joining

```
from pyspark.sql import functions as F
duplicate_keys = (customers.groupBy("customer_id").count()
                  .filter(F.col("count") > 1))
assert duplicate_keys.count() == 0
assert customers.filter(F.col("customer_id").isNull()).count() == 0
```

Repeated customer IDs in orders are normal; duplicate lookup keys are not.
Do not blindly dropDuplicates when customer records conflict.

2. Alias, join, and select explicit columns

```
o, c = orders.alias("o"), customers.alias("c")
left_orders = (o.join(c,
    F.col("o.customer_id") == F.col("c.customer_id"), "left")
  .select("o.order_id", "o.order_total", "c.customer_name",
          "c.region",
          F.col("c.customer_id").isNotNull().alias("customer_found")))
# Use "inner" instead of "left" for matched orders only.
```

Ordinary equality does not match null keys. Inspect the right lookup key.
Keep the original customer key too when investigating missing matches.

3. Reconcile rows and revenue

Actual lesson: 6 orders / 68.00 starting revenue.
Inner: 4 matched orders / 56.00. Left: 6 orders / 68.00.
Unmatched orders: 105 (customer 99) and 106 (null key), total 12.00.
BAD read-only duplicate-key demo: 8 rows / inflated 93.00.
The bad demo is NEVER persisted or used for the safe report.

Aggregate, save, and verify

4. Group the SAFE left join

```
reporting_orders = left_orders.withColumn("report_region",
  F.when(~F.col("customer_found"), F.lit("UNMATCHED"))
   .otherwise(F.col("region")))
region_summary = (reporting_orders.groupBy("report_region")
  .agg(F.count("*").alias("order_count"),
       F.count("customer_name").alias("matched_order_count"),
       F.sum("order_total").cast("decimal(18,2)").alias("revenue")))
```

count(*) counts rows; count(nullable_column) skips nulls.

Matched names are required in this lookup, not in every real dataset.

Use decimal prices; choose production precision and overflow rules deliberately.

Actual verified lesson summary

Region	Orders	Matched	Revenue
NORTH	2	2	25.00
SOUTH	1	1	22.50
UNMATCHED	2	0	12.00
WEST	1	1	8.50

CHECK: 4 buckets / 6 orders / 4 matched / revenue 68.00

5. New managed Delta table, then a persisted read

```
# table_fqn comes from the notebook's unique bt_joins_... setup.
region_summary.write.format("delta").mode("error").saveAsTable(table_fqn)
spark.sql(f"SELECT * FROM {table_fqn} ORDER BY report_region").show()
```

error refuses replacement. If write already succeeded, skip to verification.

No overwrite, append, DROP, external path, UDF, or custom Spark config.

Practice: matched-only summary has 3 real regions and total 56.00.

Companion notebook contains full schemas, safeguards, SQL and assertions.

Associate foundation: Section 3, August 2026 official guide; partial coverage.
Independent education. No certification guarantee. Fictional-data project.